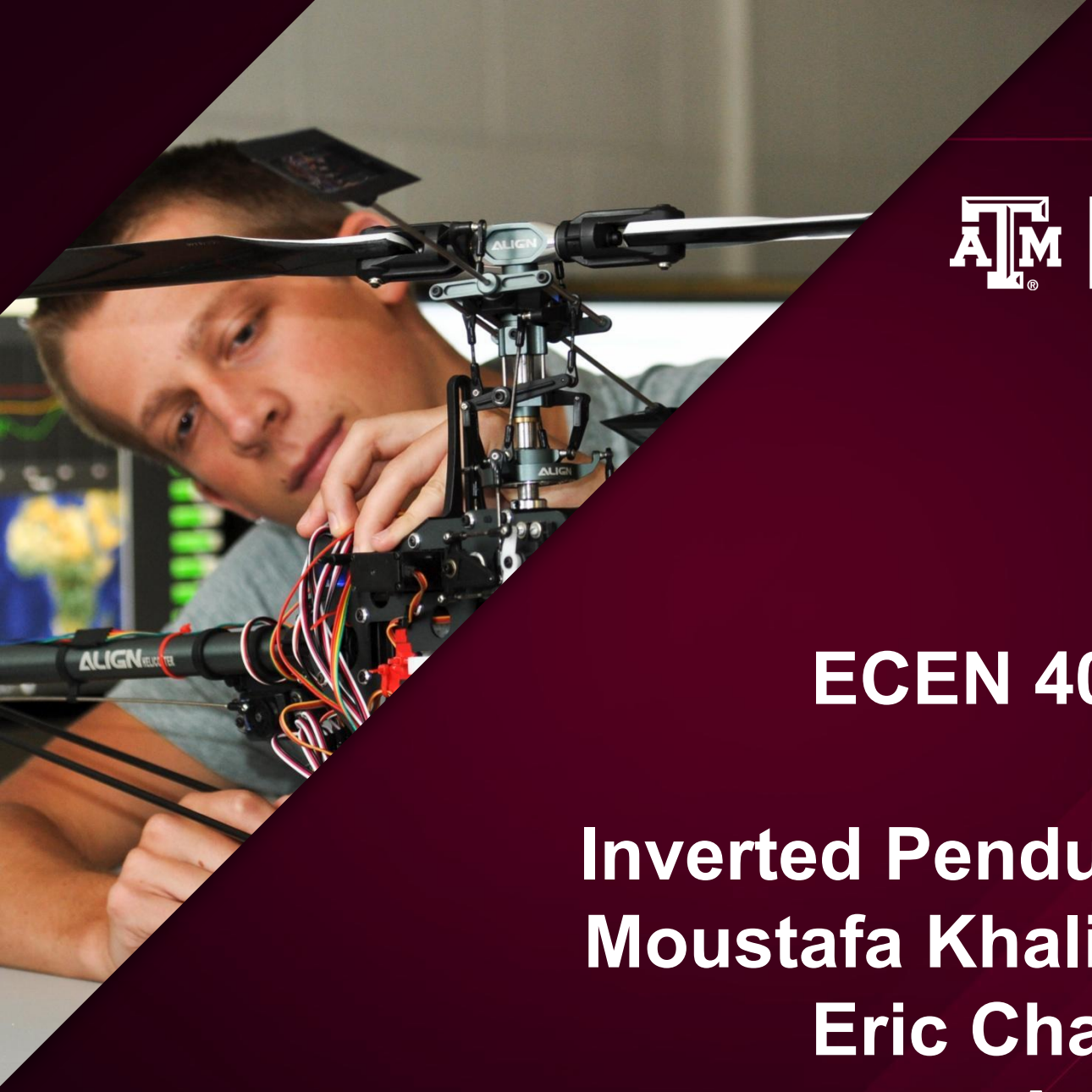


Dwight Look College of
ENGINEERING
TEXAS A&M UNIVERSITY

ECEN 403 Final Demo
Team #15
Inverted Pendulum Robot #1

Moustafa Khalil, Kaitlynn Ly,
Eric Chang, Gage Fox
Aniruddha datta

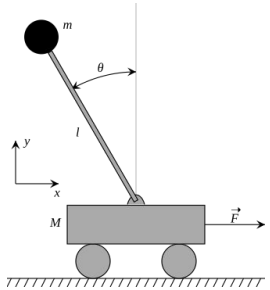
Project Summary

Problem Statement:

- Autonomous warehouse robots generally experience instability when handling unbalanced loads or making sudden maneuvers. These movements can cause tilting, making balance a key challenge.

Our Solution Will:

- Deliver an autonomous, self-navigating robot that maintains stability of an unstable payload while moving through constrained spaces.

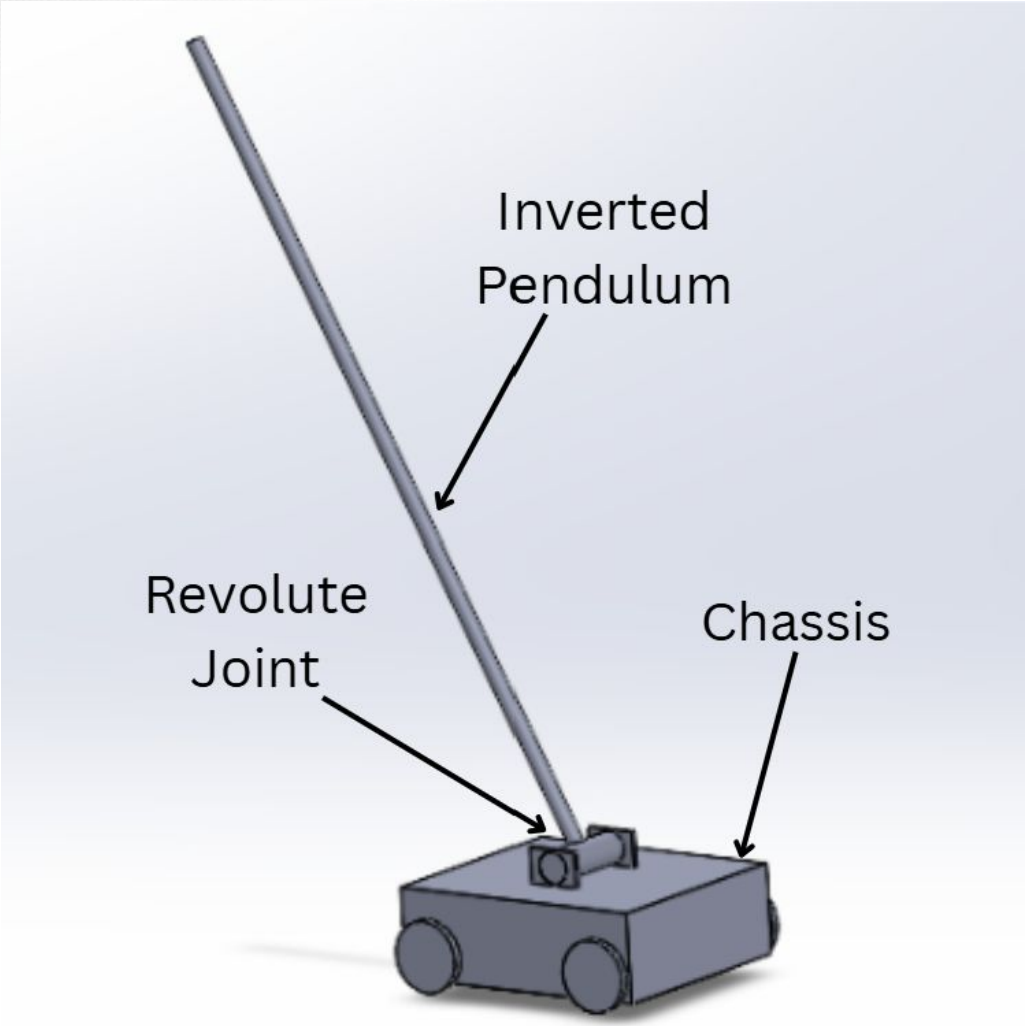


Side View

Isometric View

Controls - Moustafa Khalil

SolidWorks Model of Robot



Controls - Moustafa Khalil

- M_c - Mass of the cart/main Body
- M_p - Mass of the inverted Pendulum
- L - Length from the Pivot to Pendulum Center of Mass
- I - Inertia of the Pendulum
- r - Radius of the wheel

$\theta(t)$ - Angle of the Pendulum ($\theta = 0$, Perfectly upright)

$x(t)$ - Horizontal Position of the cart

Calculations for Controller

Energy Analysis (Lagrangian Mechanics)

$$L = T - V = \left(\text{Kinetic Energy} - \text{Potential Energy} \right)$$

Euler - Lagrange Method:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}_i}\right) - \left(\frac{\partial L}{q_i}\right) = Q_i$$

q_i = Position variable

$\dot{q}_i$ = Velocity variable

Controls - Moustafa Khalil

Kinetic Energy

$T_{cart} = \frac{1}{2} M_c \dot{x}^2$ – Cart Energy

V_p – Velocity of the Pendulums center of mass

Horizontal Velocity : $V_{px} = \dot{x} + L\dot{\theta}\cos(\theta)$

Vertical Velocity : $V_{py} = L\dot{\theta}\sin(\theta)$

$\Rightarrow V_p^2 = V_{px}^2 + V_{py}^2 = \dot{x}^2 + 2L\dot{x}\dot{\theta}\cos(\theta) + L^2\dot{\theta}^2$

$\Rightarrow T_{Pendulum} = \frac{1}{2} M_p (V_p^2) + \frac{1}{2} I \dot{\theta}^2$

Translational + Rotational

Total Kenitic Energy :

$T_{total} = \frac{1}{2}(M_c + M_p)\dot{x}^2 + M_p L \dot{x}\dot{\theta}\cos(\theta) + \frac{1}{2}(M_p L^2 + I)\dot{\theta}^2$

Potential Energy :

$V = M_p g L \cos(\theta)$

The Lagrangian (L)

$L = T - V$

$$L = \frac{1}{2}(M_c + M_p)\dot{x}^2 + M_p L \dot{x}\dot{\theta}\cos(\theta) + \frac{1}{2}(M_p L^2 + I)\dot{\theta}^2 - M_p L \cos(\theta)$$

Controls - Moustafa Khalil

Equations of Motion : Euler - Lagrange

Cart:

$$\frac{\partial L}{\partial \dot{x}} = (M_c + M_p)\dot{x} + M_p L \dot{\theta} \cos(\theta)$$

$$\Rightarrow \frac{d}{dt}\left(\frac{\partial L}{\partial \dot{x}}\right) = (M_c + M_p)\ddot{x} + M_p L \ddot{\theta} \cos\theta - M_p L \dot{\theta}^2 \sin\theta$$

$$\frac{\partial L}{\partial x} = 0 \qquad F = \frac{\tau}{r}$$

τ ← Motor Torque

r ← wheel Radius

$$\Rightarrow (M_c + M_p)\ddot{x} + M_p L \ddot{\theta} \cos(\theta) - M_p L \dot{\theta}^2 \sin(\theta) = \frac{\tau}{r}$$

Pendulum

$$\frac{\partial L}{\partial \dot{\theta}} = M_p L \dot{x} \cos(\theta) + (M_p L^2 + I)\dot{\theta}$$

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}}\right) = M_p L \ddot{x} \cos(\theta) - M_p L \ddot{x} \dot{\theta} \sin(\theta) + (M_p L^2 + I)\ddot{\theta}$$

$$\frac{\partial L}{\partial \theta} = -M_p L \dot{x} \dot{\theta} \sin(\theta) + M_p g L \sin(\theta)$$

NON Linear Eq of Motion for Pendulum

$$(M_p L^2 + I)\ddot{\theta} + (M_p L \ddot{x} \cos(\theta)) - M_p g L \sin(\theta) = 0$$

Controls - Moustafa Khalil

Linearization of non linear system

- Small Angle Approximation ($\theta \approx 0$)
  - $\cos(\theta) \approx 1$
  - $\sin(\theta) \approx 0$
  - $\dot{\theta}^2 \approx 0$

$\Rightarrow$

1) $(M_c + M_p)\ddot{x} + M_p L \ddot{\theta} = \frac{\tau}{r}$

2) $(M_p L^2 + I)\ddot{\theta} + M_p L \ddot{x} = M_p g L \theta$

State Space Representation:

Acceleration Eq:

$$\ddot{x} = \frac{M_p^2 g L^2}{I(M_c + M_p) + M_c M_p L^2}\theta + \frac{M_p L^2 + I}{r[I(M_c + M_p) + M_c M_p L^2]}\tau$$

$$\ddot{\theta} = \frac{M_p g L (M_c + M_p)}{I(M_c + M_p) + M_c M_p L^2}\theta + \frac{M_p L}{r[I(M_c + M_p) + M_c M_p L^2]}\tau$$

$$\text{Den} = I(M_c + M_p) + M_c M_p L^2$$

$$\dot{x} = Ax + Bu$$

Controls - Moustafa Khalil

$x = [x, \dot{x}, \theta, \dot{\theta}]^T$ – State space form

$$\begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\theta} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{-M_p^2 g L^2}{Den} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{M_p g L (M_c + M_p)}{Den} & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{M_p L^2 + I}{(Den) r} \\ 0 \\ \frac{-M_p L}{(r)(Den)} \end{bmatrix} \tau$$

LQR Controller Design

K (LQR Gains) stabalizes the system

The cost Function (J)

$$J = \int_0^\infty (x^T Q x + u^T R u)\, dt$$

- x – State Error
- u – Control Input
- Q – State Cost Matrix (Penalty on Error)
- R – Input Cost Matrix (Penalty on Effort)

$u = -Kx$

use Algebraic Riccati Equation (ARE)

$$A^T P + PA - PBR^{-1}B^T P + Q = 0$$

- solve for P via matlab

Then

$$K = R^{-1} B^T P$$

```
M = 3.5;        % Mass of Cart [kg]
m = 1.3;        % Mass of Pendulum Rod [kg]
L = 0.505668;     % Length to Center of Mass [m]
r = 0.0355;     % Wheel Radius [m] (35.5 mm)
g = 9.80665;      % Gravity [m/s^2]

% Moment of Inertia (Point Mass assumption from Simscape)
I = 0;
```

```
%% 1. System Matrices (The Linearized Model)
% State Vector X = [x; x_dot; theta; theta_dot]

% Denominator term
den = I*(M+m) + M*m*L^2;

% Matrix A (System Dynamics)
A = [0, 1, 0, 0;
     0, 0, -(m^2*g*L^2)/den, 0;
     0, 0, 0, 1;
     0, 0, (m*g*L*(M+m))/den, 0];

% Matrix B (Input Effectiveness - Torque)
% We convert Force to Torque using (1/r) inside the matrix
B = [0;
     (m*L^2 + I)/(den*r);
     0;
     -(m*L)/(den*r)];

C = eye(4);
D = [0;0;0;0];
```

```
%% 2. LQR Cost Weights (Tuning)
% Q = Penalties on [Position, Speed, Angle, AngularVelocity]

Q = diag([0.1,  0.1,  500,  1]);
R = 1; % Penalty on Motor Effort
```

```
%% 3. Calculate Optimal Gains (K)
K = lqr(A, B, Q, R);

fprintf('-----------------------------------------------\n');
fprintf('The LQR Gains are:\n');
fprintf('K = [ %.4f,  %.4f,  %.4f,  %.4f ]\n', K(1), K(2), K(3), K(4));
fprintf('-----------------------------------------------\n');
```

```
-----------------------------------------------
>> LQR
-----------------------------------------------
The LQR Gains are:
K = [ -0.3162,  -1.2764,  -25.3829,  -2.6936 ]
-----------------------------------------------
```

- This script calculates the optimal Linear Quadratic Regulator (LQR) gains to stabilize the inverted pendulum by defining its physical system matrices and minimizing the cost function for state errors and control effort.

```
%% PID Calculation Script for Mobile Inverted Pendulum
% ----------------------------------------------------
% 1. System Parameters
M = 3.5; m = 1.3; L = 0.505; r = 0.0355; g = 9.81;
I = 0;
den = I*(M+m) + M*m*L^2;

% 2. State Space Matrices (A, B, C, D)
A = [0 1 0 0; 0 0 -(m^2*g*L^2)/den 0; 0 0 0 1; 0 0 (m*g*L*(M+m))/den 0];
B = [0; (m*L^2 + I)/(den*r); 0; -(m*L)/(den*r)];
C = [1 0 0 0]; % Controls Position (State 1)
D = 0;

% 3. Defining the LQR Inner Loop (Stabilization)
% INSERT LQR GAINS HERE:
K_LQR = [ 0, 0, 25.3830, 2.8 ];
% (Note: use 0 for position/velocity because PID handles that now)

% 4. Create the "Stabilized Plant"
% The PID sees the robot + LQR as one single "Black Box"
A_closed = A - B * K_LQR;
sys_inner = ss(A_closed, B, C, D);

% 5. Design the PID Controller
% "Rise time of about 2 seconds" (Smooth movement)
target_bandwidth = 1.5; % Speed of response (rad/s)

% Calculate PID options
opts = pidtuneOptions('PhaseMargin', 60, 'DesignFocus', 'reference-tracking');
[C_pid, info] = pidtune(sys_inner, 'PID', target_bandwidth, opts);
```

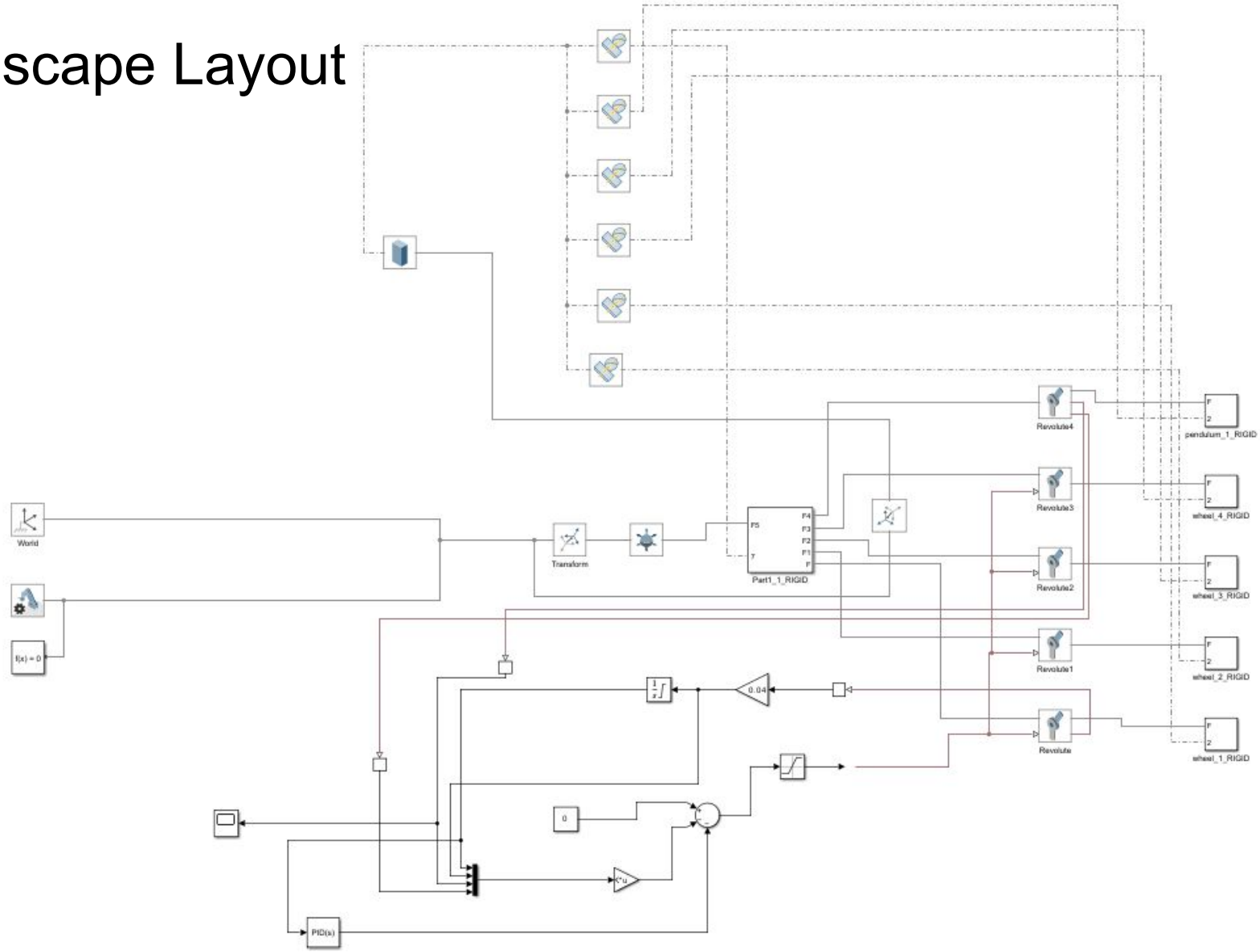
```
% 6. Display the Calculated Values
Kp = C_pid.Kp;
Ki = C_pid.Ki;
Kd = C_pid.Kd;

fprintf('-----------------------------------------\n');
fprintf('CALCULATED PID VALUES (Theoretical):\n');
fprintf('Kp = %.5f\n', Kp);
fprintf('Ki = %.5f\n', Ki);
fprintf('Kd = %.5f\n', Kd);
fprintf('-----------------------------------------\n');
```

```
-----------------------------------------
>> PID_calc
-----------------------------------------
CALCULATED PID VALUES:
Kp = -0.19793
Ki = -0.00260
Kd = -3.76975
-----------------------------------------
```

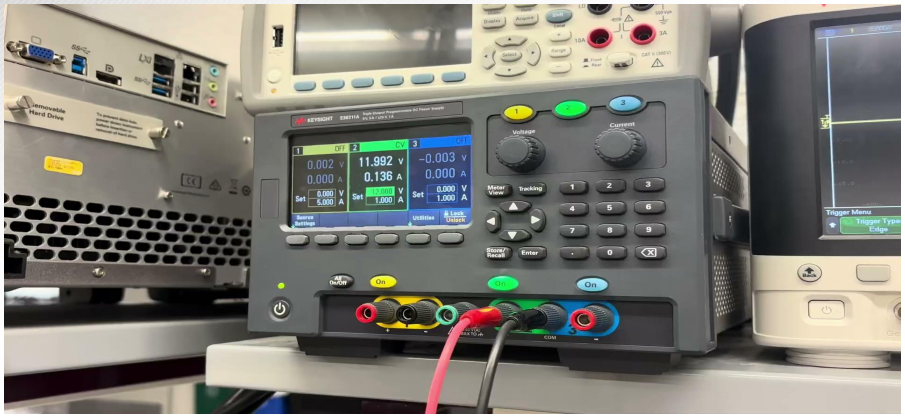
- This script tunes a Proportional-Integral-Derivative (PID) controller to regulate the robot's position by wrapping around the inner LQR stabilization loop and optimizing the gains for a smooth 2-second response time.

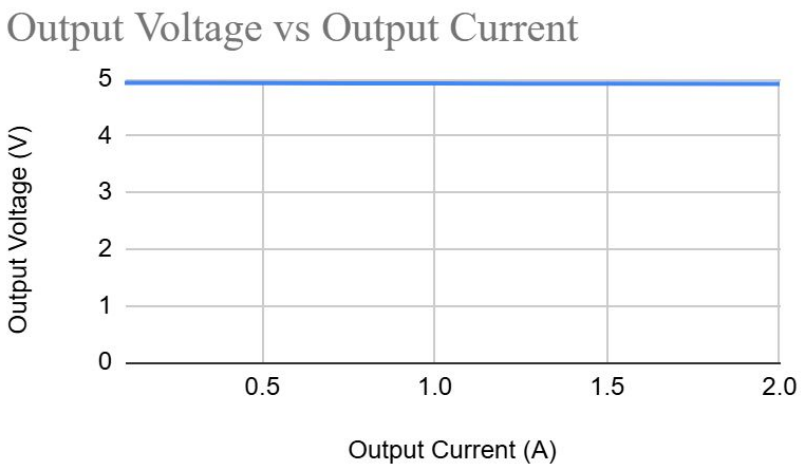
Controls - Moustafa Khalil

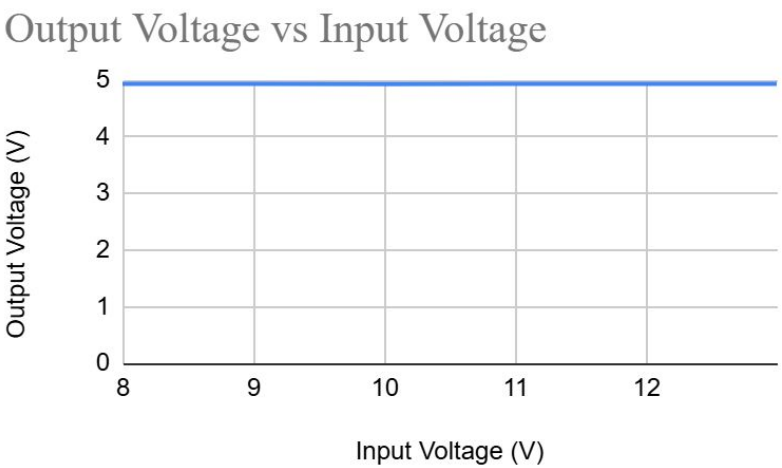
Simscape Layout



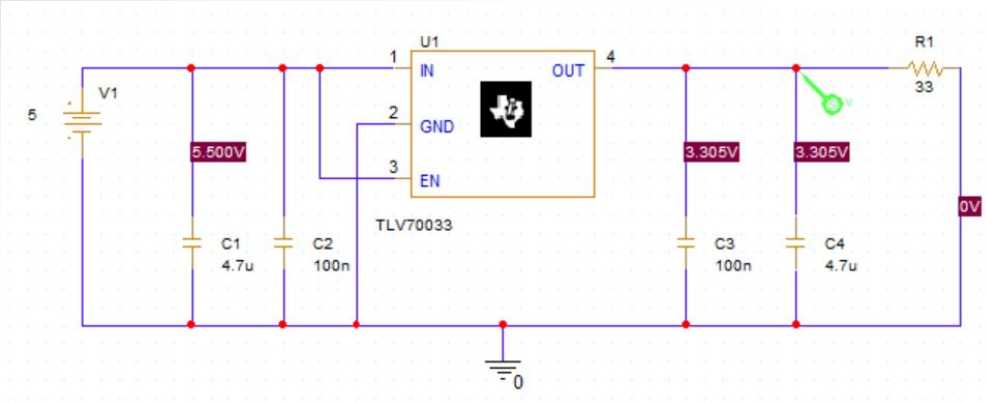
Power & Mechanical - Kaitlynn Ly

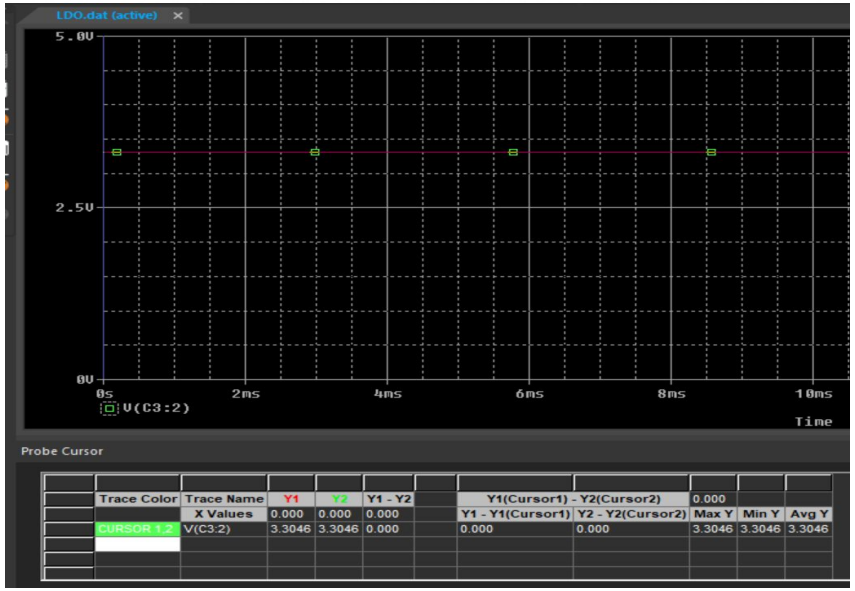


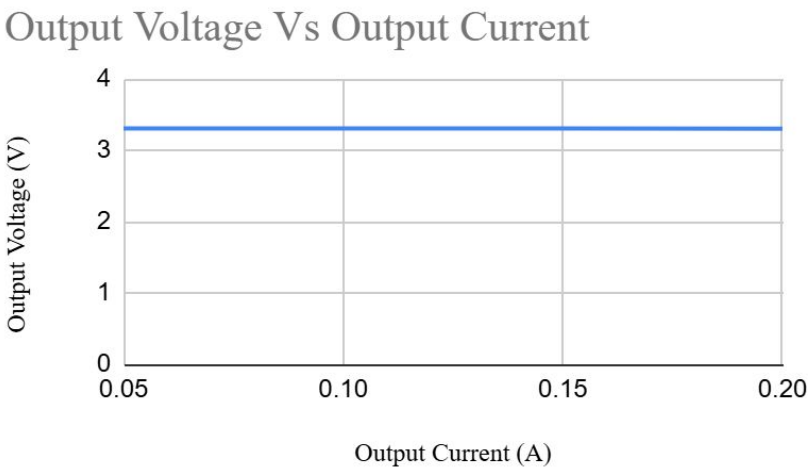


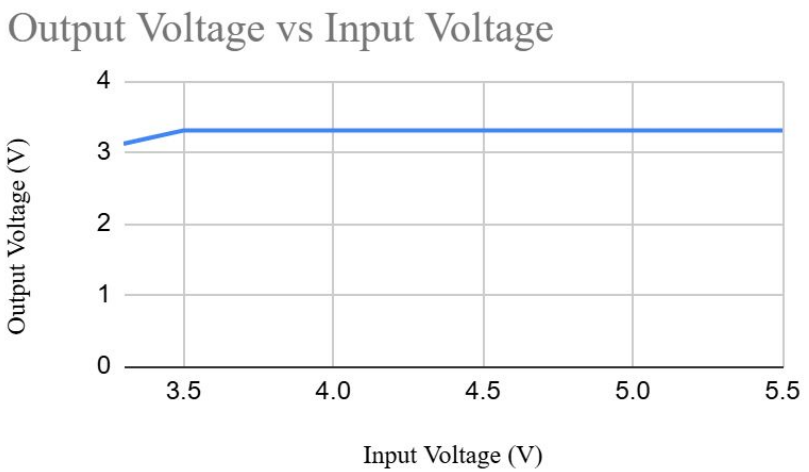


Power & Mechanical

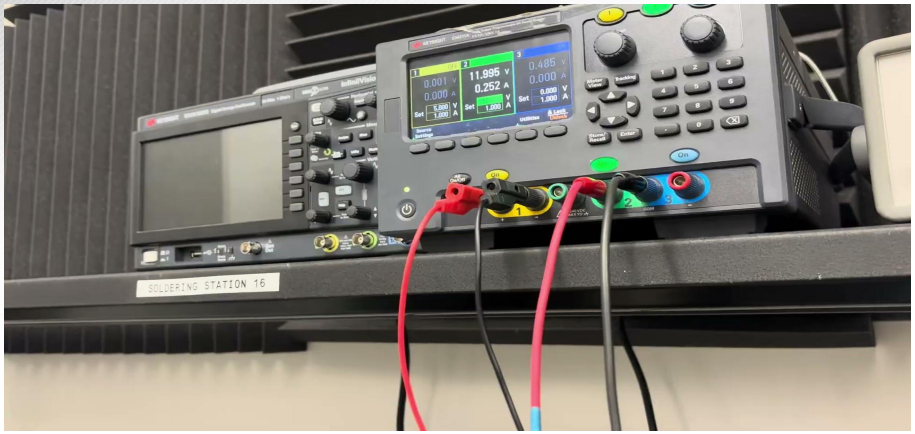


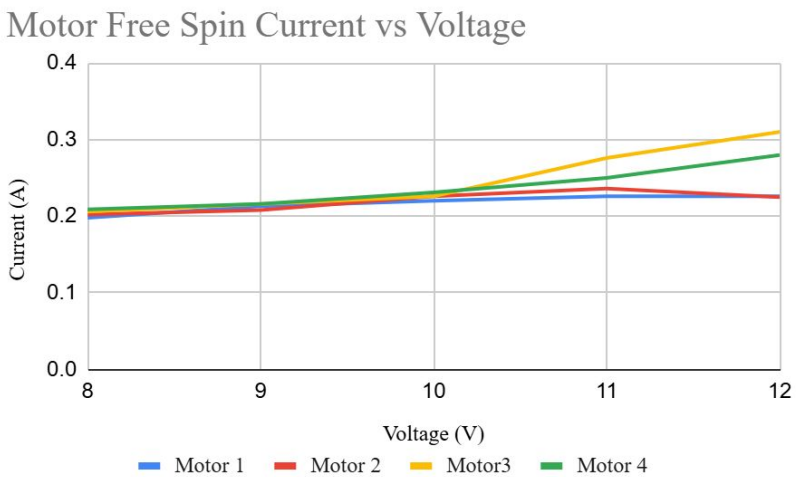


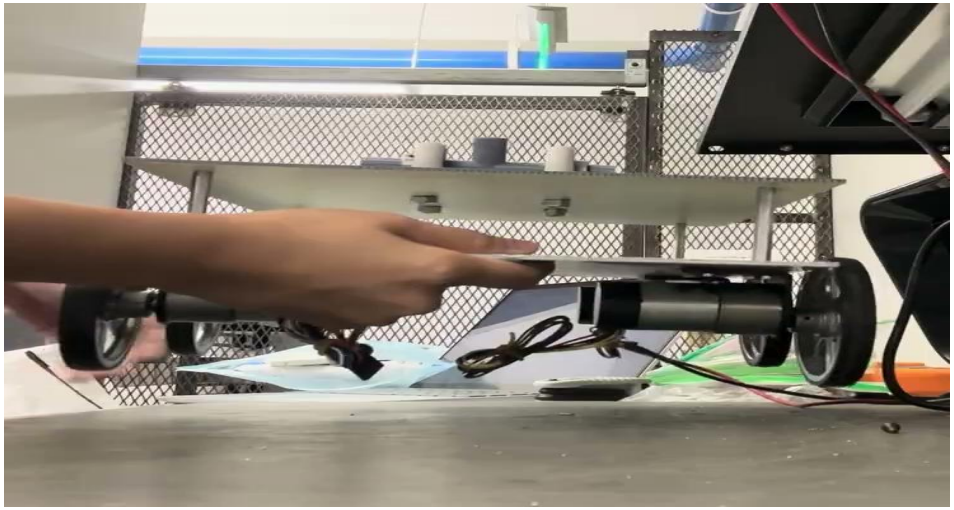


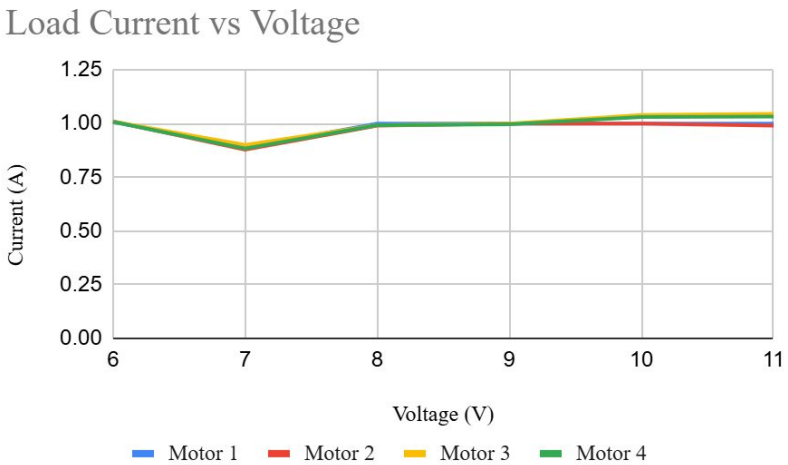


Power & Mechanical

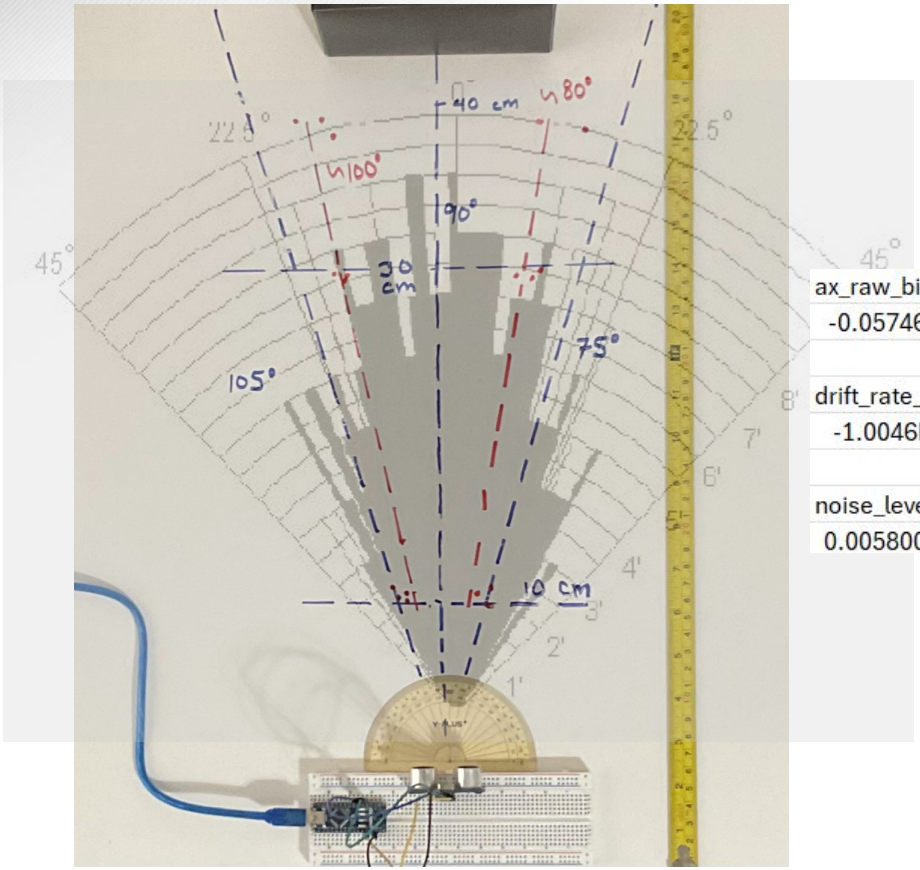








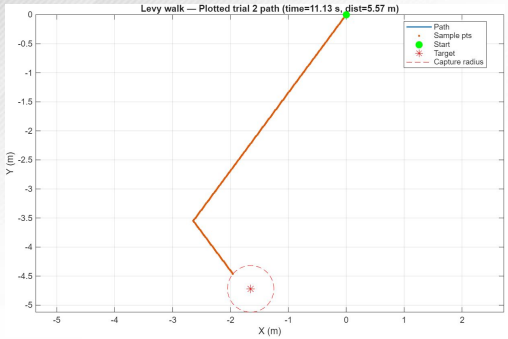
Sensors, Navigation, GUI - Eric Chang

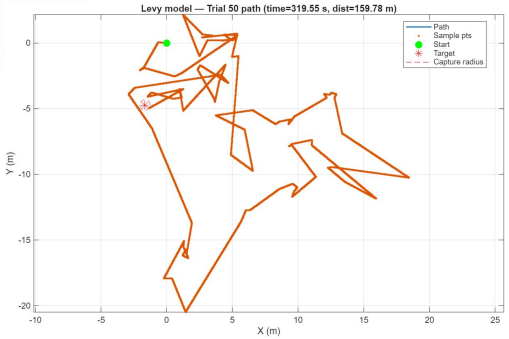


ax_raw_bias	ay_raw_bias	az_raw_bias	gx_raw_bias	gy_raw_bias	gz_raw_bias
-0.05746153	0.026018076	10.01612952	0.520786678	-0.15119873	-0.513364314
drift_rate_ax	drift_rate_ay	drift_rate_az	drift_rate_az	drift_rate_az	drift_rate_az
-1.0046E-06	-9.33491E-07	2.26452E-06	1.71288E-05	-3.9321E-06	-1.32619E-06
noise_level_ax	noise_level_ay	noise_level_az	noise_level_gx	noise_level_gy	noise_level_gz
0.005800029	0.005504147	0.004918569	0.035259358	0.02669854	0.033151445

Random Walk Efficiency

Lévy

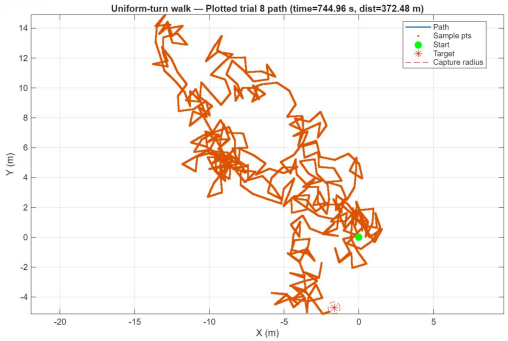


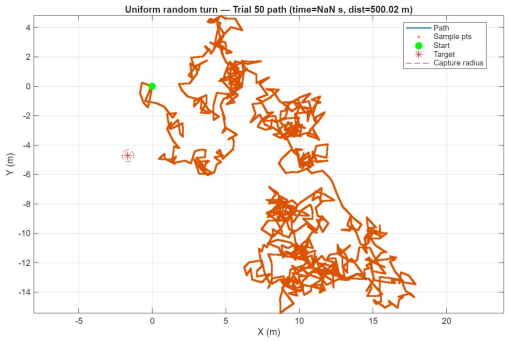


mean PER: 0.0665

P/F: 15/35

Uniform-turn

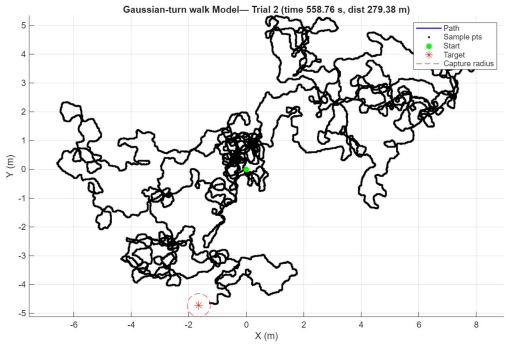


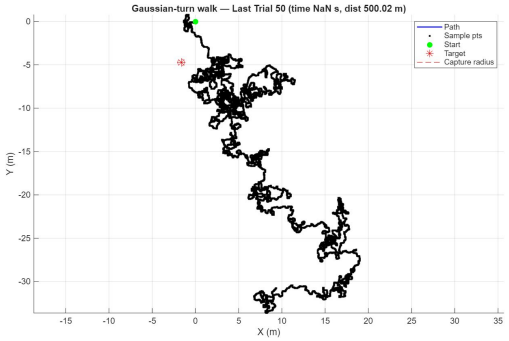


mean PER: 0.0406

P/F: 13/37

Gaussian-turn

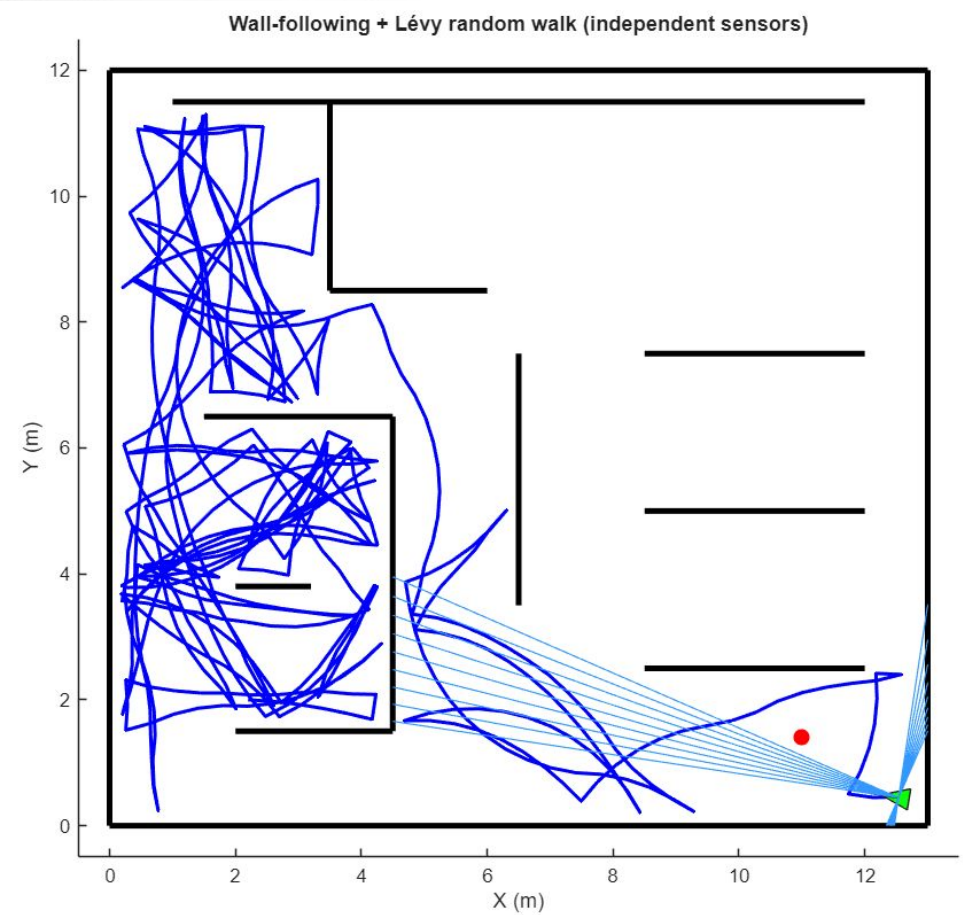




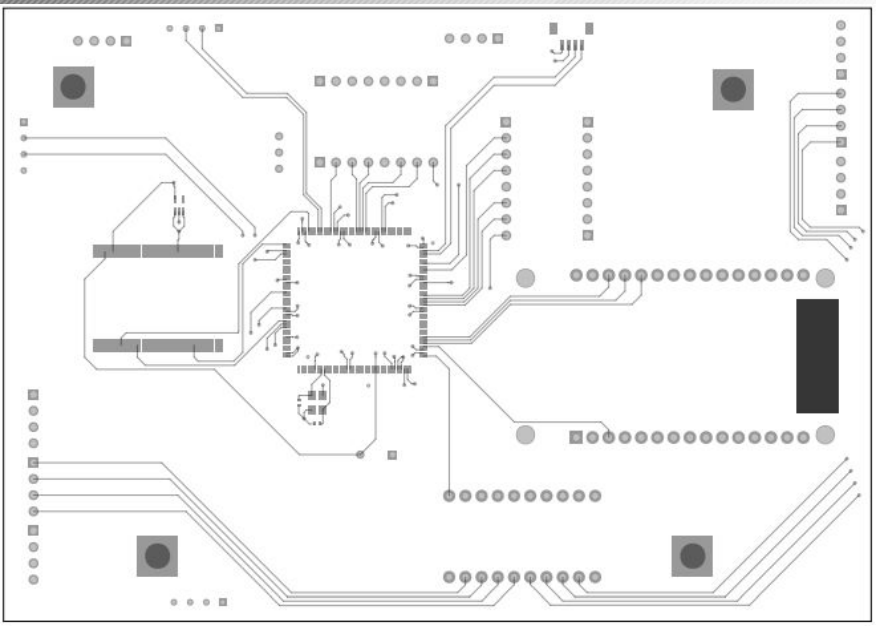
mean PER: 0.0406

P/F: 12/38

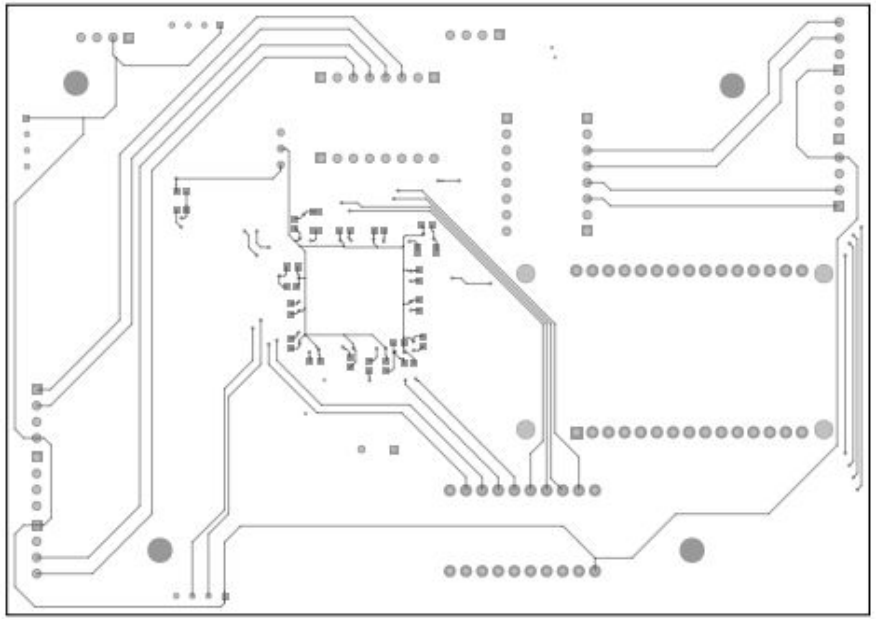
Path Planner



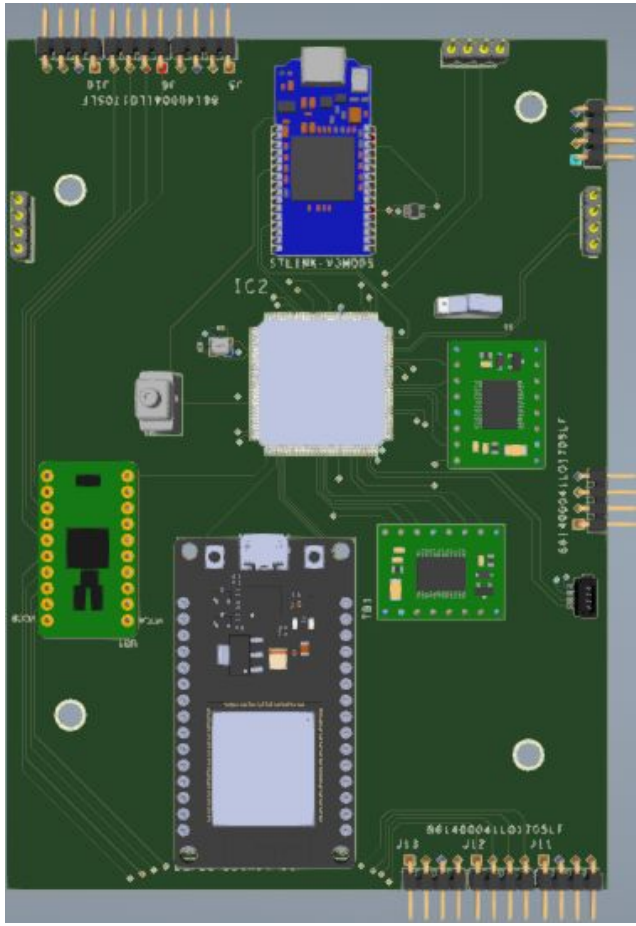
Logic PCB - Gage Fox



Top

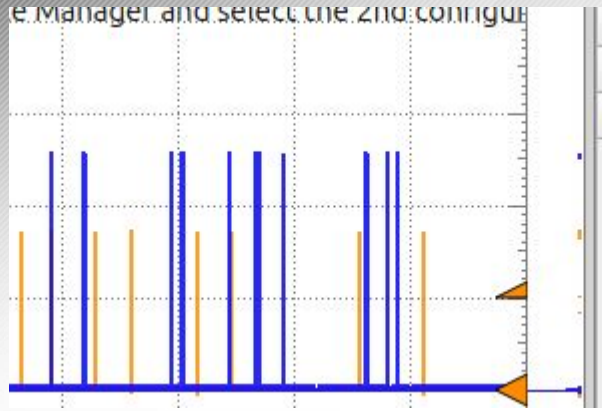


Bottom



3D-Top

MCU to Ultrasonic Sensor Comms - Gage Fox



	Name	Value
C2	Maximum	5.1486 V
C2	High	5.0912 V

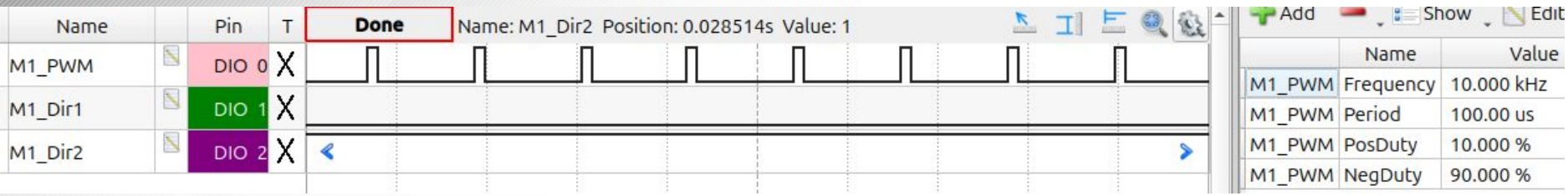
Maximum Echo Voltage

	Name	Value	Count	Maximum	Deviation
Sonar_Trig	Period	60.996 ms	2	60.996 ms	99.999 ns
Sonar_Trig	PosWidth	10.867 us	3	11.000 us	94.281 ns
Sonar_Trig	NegWidth	60.985 ms	2	60.985 ms	0.00000 s
Sonar_Echo	PosWidth	5.4612 ms	3	5.4694 ms	11.324 us
Sonar_Echo	NegWidth	55.539 ms	2	55.551 ms	12.060 us

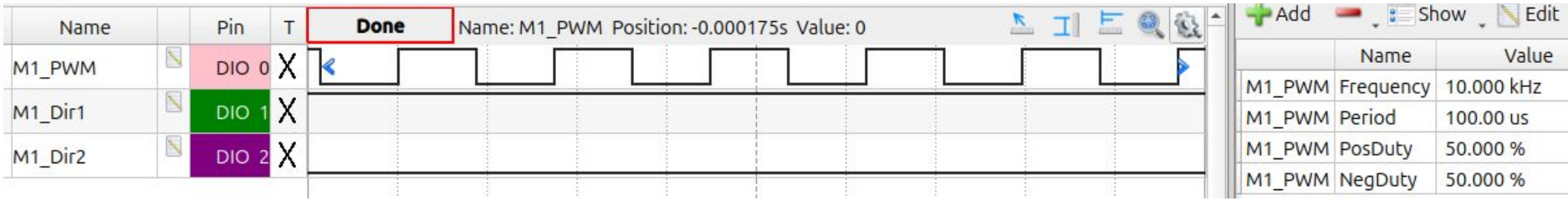
Trigger and Echo Signal Widths and Periods

Trigger and Echo Relationship

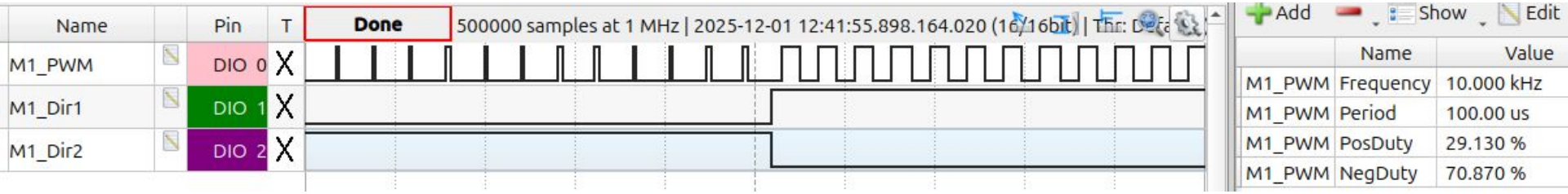
MCU to Motor Comms - Gage Fox



Timer 1 Channel 1 PWM - 10% Duty Cycle @ 10kHz

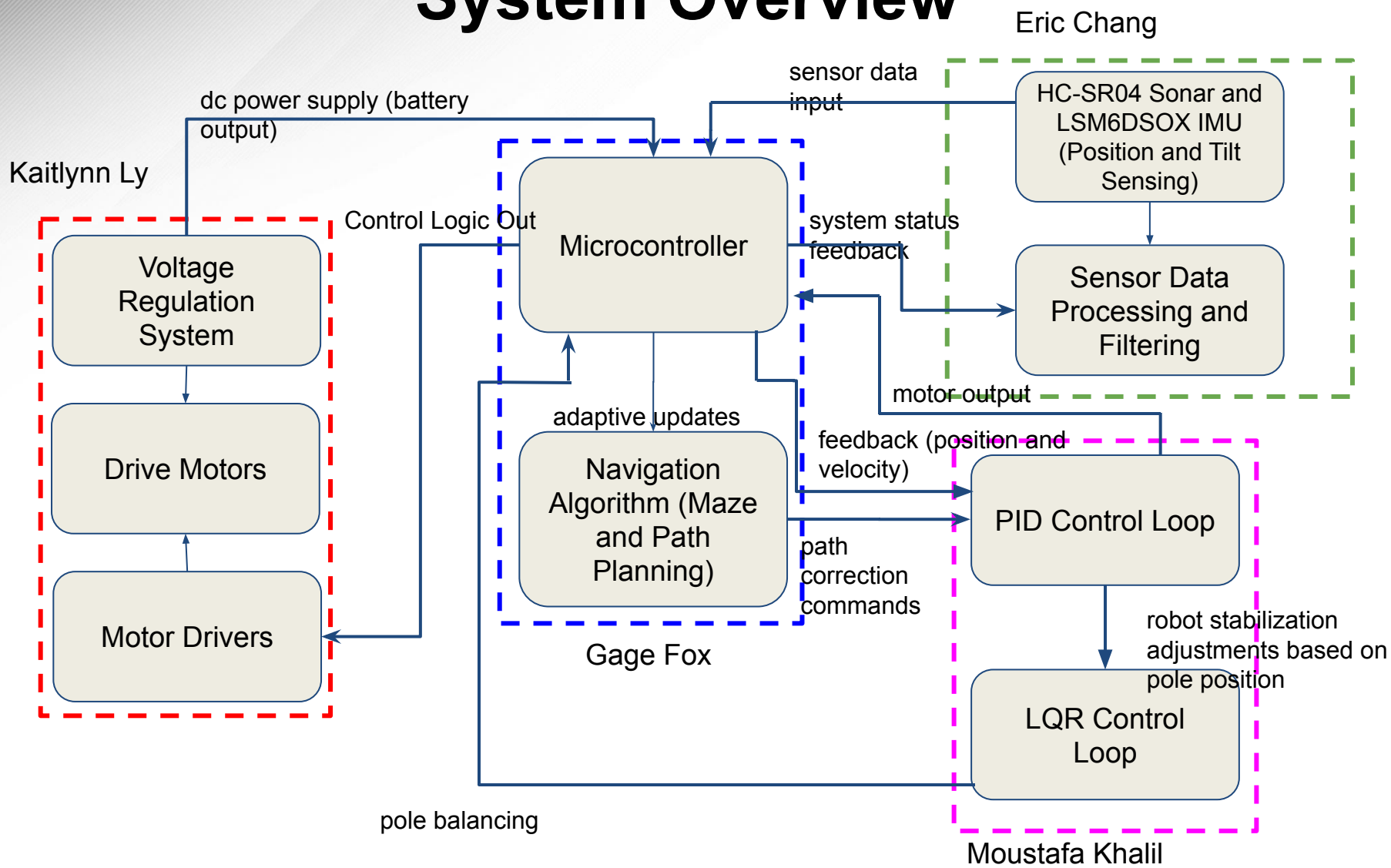


Timer 1 Channel 1 PWM - 50% Duty Cycle @ 10kHz



Timer 1 Channel 1 PWM - 10->50%Duty Cycle @ 10kHz

System Overview



Validation Plan

Task	Specifications and Success Criteria	Status	Responsible Engineer(s)
Logic Calculation Rate	Validate through timing tests on MCU execution cycle; must complete calculations in < 50ms.	Tested	Gage
Data Transfer Rates	Measure message latency with time logs; must be < 100 ms difference.	Tested	Gage
Sensor Compatibility	Ensure sensor logic is compatible with MCU designated MCU pins.	Tested	Gage
Sonar Read Distance	Validate using logged ultrasonic data (CSV) and IMU; confirm ≥95% obstacle detection at distances ≥2 cm and ±3 cm accuracy within 0.2-1.0 m	Tested	Eric
Sensor Field-of-view Testing	Verify that the ultrasonic sensor coverage meets the ±15° design requirements	Tested	Eric
IMU Orientation Error	Verify that orientation measurements have ≤ 2° steady-state error and minimal drift	Tested	Eric
Telemetry/GUI Latency	Mean latency < 200 ms with minimal packet loss	Tested	Eric
Path Efficiency Ratio	Determine the most efficient navigation method by identifying which of the random-walk methods produces the greatest PER under similar conditions	Tested	Eric
Motor Power Consumption	Measure motor current at multiple voltages under both free-spin and load conditions; confirm total motor power draw is ~40–45W	Tested	Kaitlynn
Buck Converter Regulation	Record output voltage under different loads; verify 5V stability	Tested	Kaitlynn
LDO Validation	Measure regulated output with oscilloscope; confirm 3.3V stability	Tested	Kaitlynn
Control Algorithm Validation	Validate stability across various operating conditions and confirm tuned gains achieve target response time	Tested	Moustafa
Initial Condition Response (LQR)	In simulation, initialize the pendulum at a -10° offset (without PID input) and observe the angle vs. time graph as it stabilizes.	Tested	Moustafa
Step Response Tracking (PID)	Validate tracking performance by applying a 1.0 m step input; confirm rise time is < 2.5 s, overshoot is < 10%	Tested	Moustafa

Thank You!

Inverted Pendulum Robot 1

Any Questions?